
Chap 7, 8: Scheduling

Introduction

- **Multiprogramming**

- Multiple processes in the system with one or more processors
- Increases processor utilization by organizing processes so that the processor always has one to execute
- Resource management

- Resources for **time sharing**

- Multiple processes use a resource in a time-shared manner

- **Processor**



- Process scheduling: Allocates processor time slots to processes

- Resources for **space sharing**

- Partition a resource and let each process use the partitions

- **Memory**

Mem



Goals of Scheduling

- **Goals of process scheduling**

- Improving system performance

- **Typical performance indices**

- **Turnaround time**: amount of time to execute a particular process

- $T_{\text{turnaround}} = T_{\text{completion}} - T_{\text{arrival}}$

- **Response time**: amount of time it takes to start responding

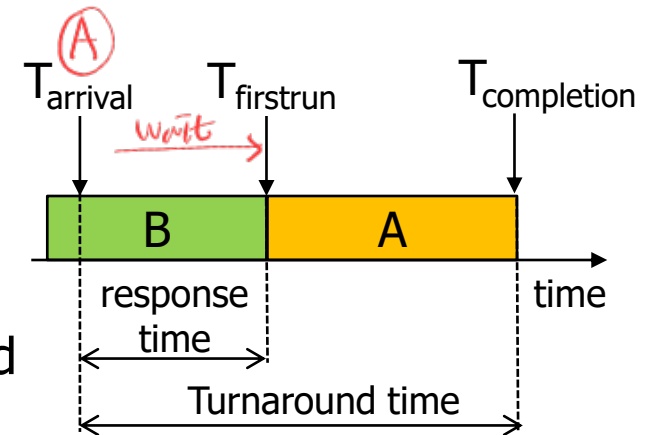
- $T_{\text{response}} = T_{\text{firstrun}} - T_{\text{arrival}}$

- **Throughput**: number of processes completed per time unit

- **Utilization**: Percentage of time that the resource is busy during a given interval

- Fairness, Predictability, Etc

- Each system selects a scheduling policy with the consideration on the performance indices/for its application domain
각 운영체제는 해당 domain에서 중요하게 여기는 performance 지표를 고려해 scheduling policy를 선택



특정 작업이 너무
오래 실행되는 것은
부담

→ real-time system에서 사용
(작업이 언제 시작되고 끝날지 예측)

Scheduling Policies

- **Preemptive/non-preemptive scheduling**

- Non-preemptive scheduling (Cooperative scheduling)

- Process uses the CPU until it voluntarily releases it
(eg. for system call) *yield/exit*

*process가 스스로 release할 때까지 CPU 사용
강제로 빼앗지 않음)*

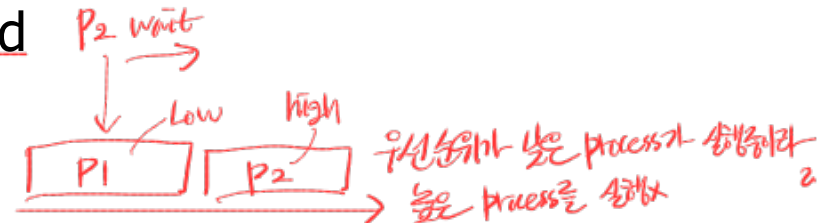
- No preemption

- Pros

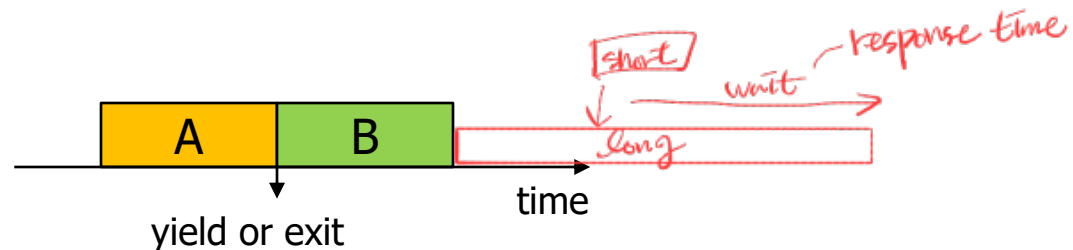
- Low context switch overhead

- Cons

- Frequent priority inversions



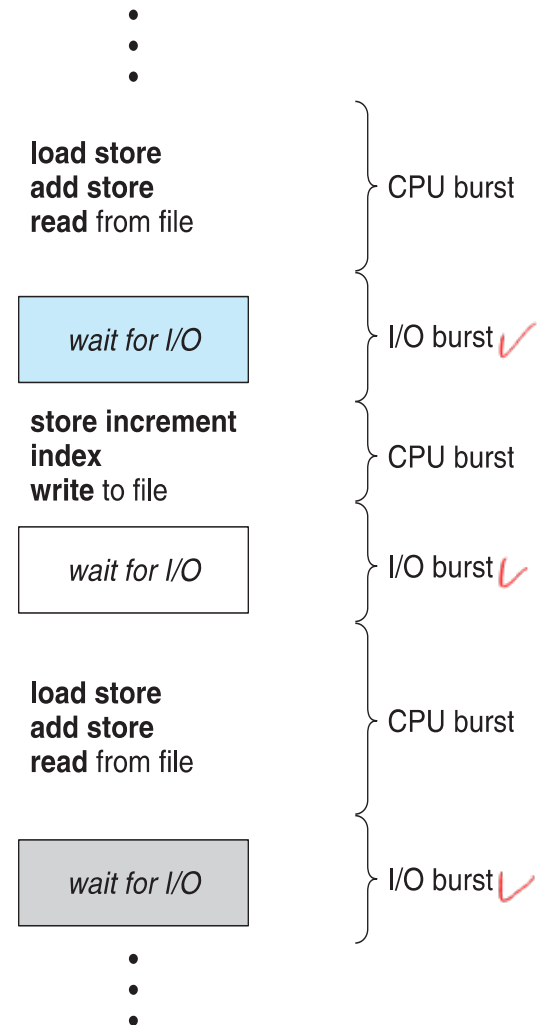
- May result in longer mean response time



Terminologies

- **CPU burst vs. I/O burst**

- Process execution consists of a cycle of CPU execution and I/O wait
- CPU burst *CPU를 사용하는 시간*
 - Each cycle of CPU execution
- I/O burst *process가 I/O를 기다리는 시간 CPU는 다른 process*
 - Each cycle of I/O wait
- Burst time is an important factor(criteria) for scheduling algorithms



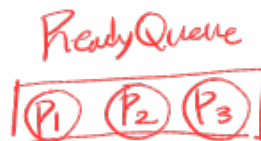
Scheduling Schemes

- **FIFO, FCFS (First-Come First Service)**
- **SJF (Shortest Job First)**
- **STCF (Shortest Time-to-Completion First)**
- **RR**
- **Priority**
- **MLFQ**

Scheduling Schemes

- **FCFS(First-Come-First-Service) scheduling**

- Non-preemptive scheduling
- Scheduling criteria
 - Arrival time (at the ready queue)
 - Faster arrival time process first



- High resource utilization *non-preemptive 이걸 이용하지 않음*
- Adequate for batch systems, not for interactive systems
- Disadvantages

- **Convoy effect**

- short process behind long process
- Consider one CPU-bound and many I/O-bound processes

- Longer mean response time

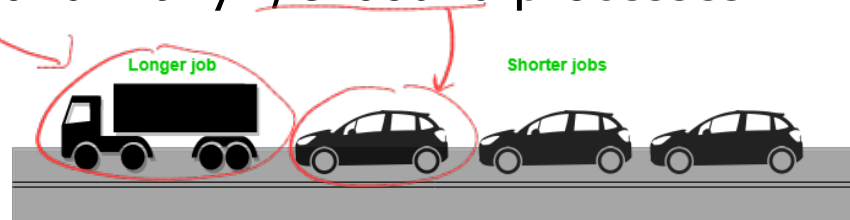


Figure - The Convoy Effect Visualized

First-Come, First-Served (FCFS) Scheduling

<u>Process</u>	<u>Burst Time</u>
P_1	24
P_2	3
P_3	3

- Suppose that the processes arrive in the order: P_1, P_2, P_3
The Gantt Chart for the schedule is:



- Turnaround time for $P_1 = 24$; $P_2 = 27$; $P_3 = 30$
- Average Turnaround time: $(24 + 27 + 30)/3 = 27$

FCFS Scheduling (Cont.)

Suppose that the processes arrive in the order:

P_2, P_3, P_1

- The Gantt chart for the schedule is:



- Turnaround time for $P_1 = 30$; $P_2 = 3$; $P_3 = 6$
- Average Turnaround time: $(30 + 3 + 6)/3 = 13$
- Much better than previous case

Scheduling Schemes

- **SJF (Shortest Job First) scheduling**

- Non-preemptive scheduling
- Scheduling criteria
 - Burst time (CPU A&A)
 - Shortest next CPU burst time first scheduling
- Pros
 - Gives minimum average waiting time for a given set of processes
 - Minimizes the number of processes in the system
 - Reduces the size of the ready queue
 - Reduces the overall space requirements
 - Fast responses to many processes

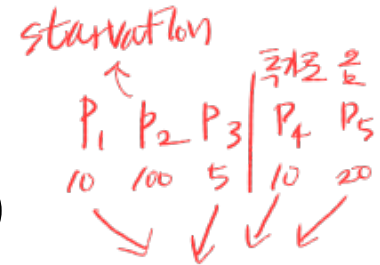
ready queue
RQ
T 0 0 0 0 0 0 0
↓ shortest

Scheduling Schemes

- **SJF (Shortest Job First) scheduling**

- Cons

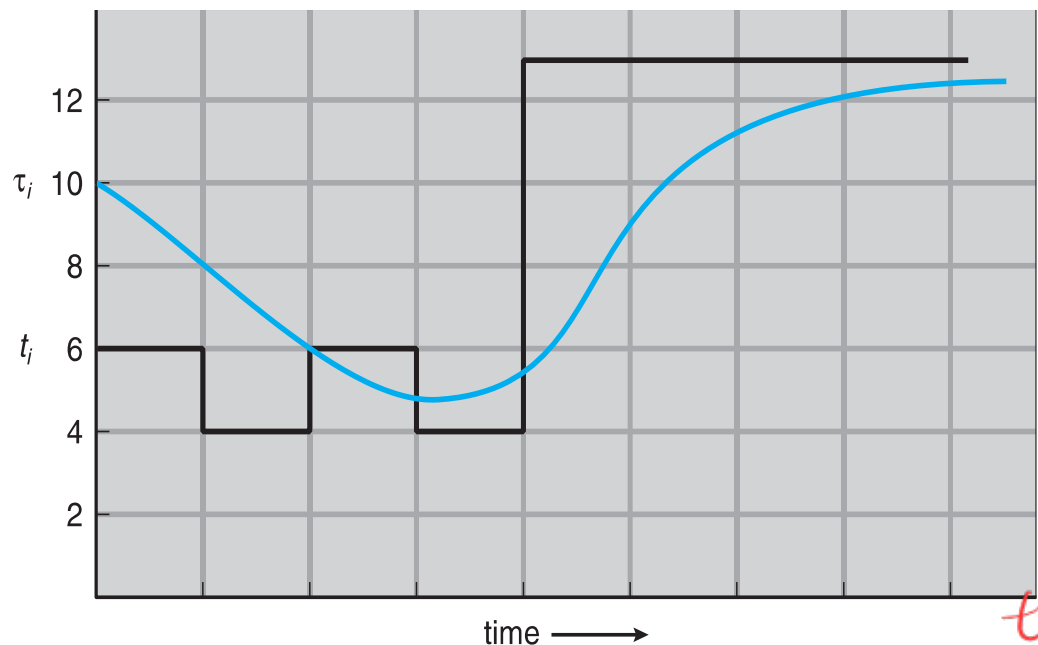
- Starvation, indefinite postponement(blocking)
 - Long burst-time processes → *장시간 프로세스*
 - Can be solved by aging
 - No way to know the length of the next CPU burst for each process *프로세스와 다음 CPU burst를 미리 알 수 없음*
 - It is necessary to have a scheme for burst time estimation
 - Estimation by exponential average *past history 사용*
 1. t_n = actual length of n^{th} CPU burst
 2. τ_{n+1} = predicted value for the next CPU burst
 3. $\alpha, 0 \leq \alpha \leq 1$
 4. Define : $\tau_{n+1} = \alpha t_n + (1 - \alpha)\tau_n$. Commonly, α set to $1/2$
recursive



Scheduling Schemes

- SJF (Shortest Job First) scheduling**

$$\tau_{n+1} = \left(\alpha t_n + (1 - \alpha) \alpha t_{n-1} \right) + \dots + (1 - \alpha)^j \alpha t_{n-j} + \dots + (1 - \alpha)^{n+1} \tau_0$$

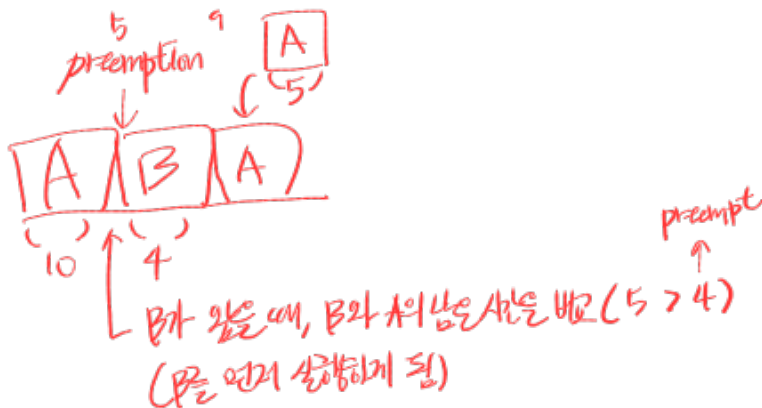


CPU burst (t_i)	6	4	6	4	13	13	13	...	
"guess" (τ_i)	10	8	6	6	5	9	11	12	...

Scheduling Schemes

- **STCF (Shortest Time-to-Completion First) scheduling**

- Variation of SJF scheduling (preemptive SJF)
- Preemptive scheduling
 - Preempt current running process when another process/with shorter remaining CPU burst time arrives at the ready queue
- Cons
 - Burst time estimation overhead as in SJF
 - Overhead for tracing remaining burst time
 - High context switching overhead



Example of STCF

- Now we add the concepts of varying arrival times and preemption to the analysis

<u>Process</u>	<u>Arrival Time</u>	<u>Burst Time</u>
P_1	0	8
P_2	1	4
P_3	2	9
P_4	3	5

turnaround time

$$(17-0)=17$$

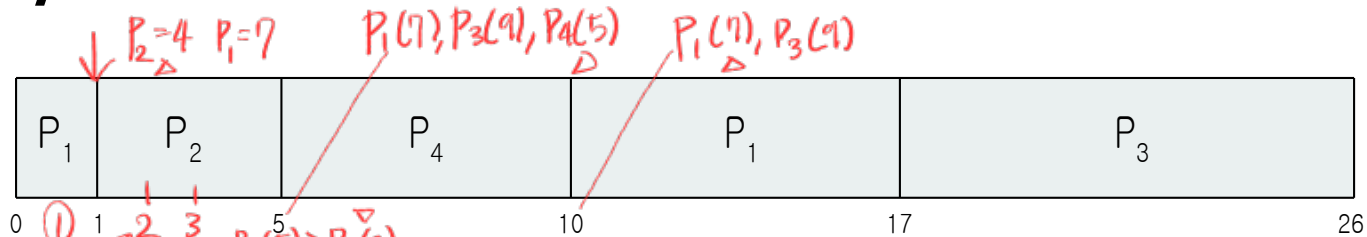
$$(5-1)=4$$

$$(26-2)=24$$

$$(10-3)=7$$

$$52/4=13$$

- Preemptive SJF Gantt Chart**



- Average Turnaround time = $[(17-0)+(5-1)+(26-2)+(10-3)]/4 = 52/4 = 13$

$$P_3(9) > P_2(4)$$

간격당 중에 짧은 작업이 있으면 preemption,
→ short response time

A New Metric: Response Time

- At time-shared machines, users would sit at a terminal and demand interactive performance from the system. *response time이 중요한 것*
- Response time:** the time from when the job arrives in a system to the first time it is scheduled
 - $T_{\text{response}} = T_{\text{first run}} - T_{\text{arrival}}$
first run *시작*

A, B, and C arrive at the same time, *같은 CPU burst*

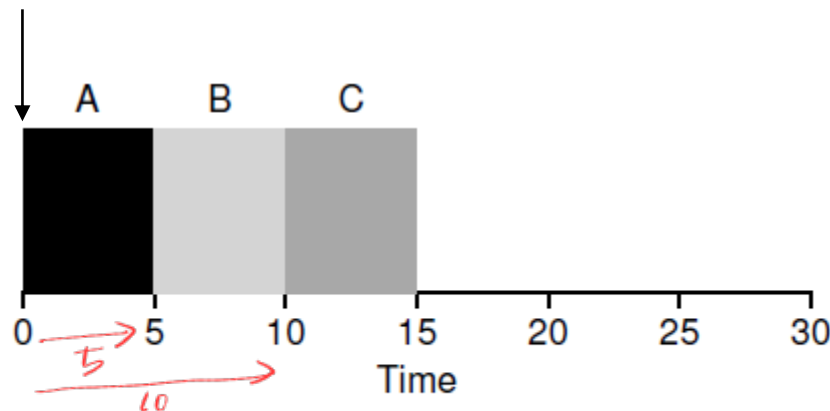


Figure 7.6: SJF Again (Bad for Response Time)

Scheduling Schemes

- **RR (Round-Robin) scheduling**
 - Preemptive scheduling
 - Scheduling criteria
 - Arrival time (at the ready queue)
 - Faster arrival time process first
 - Time slice (scheduling quantum) for each process
 - System parameter
 - The (running) process that has exhausted his time slice ^{1s, 2s} releases the CPU and goes to the ready state (timer runout)
 - Prevents monopoly of the CPU by a process
 - High context switching overhead due to preemptions
 - Adequate for interactive/time-sharing system

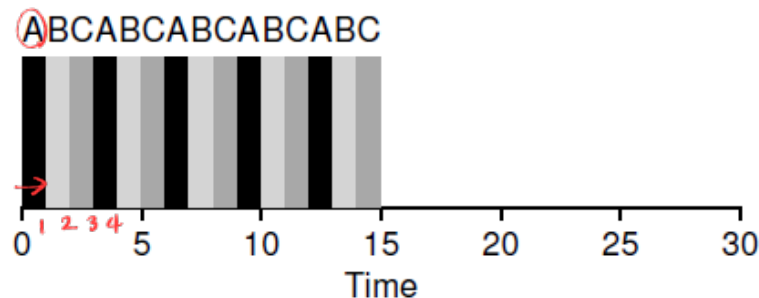


Figure 7.7: Round Robin (Good for Response Time)

Scheduling Schemes

- **RR (Round-Robin) scheduling**

- Performance of the RR scheme depends heavily on the size of the time slice *time slice는 parameter인 performance가 다함*

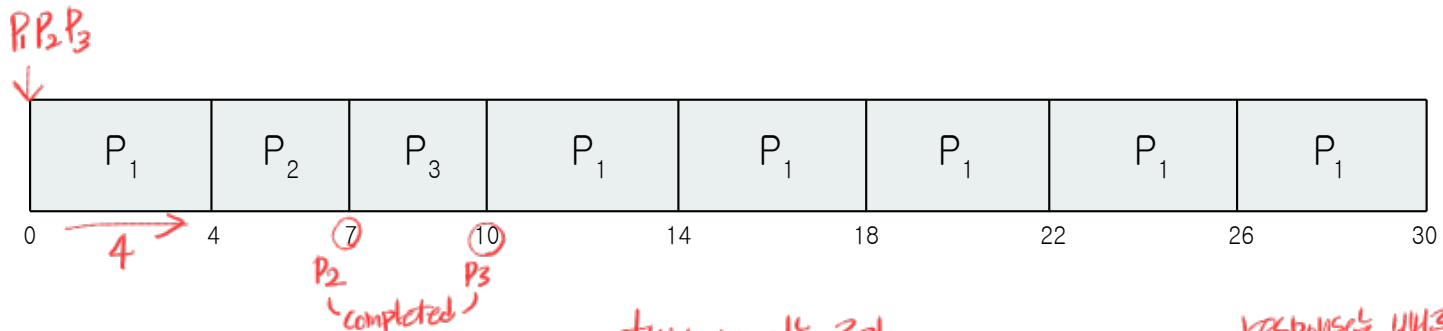
- Very large (infinite) time slice → FCFS
- Very small time slice → processor sharing
 - Appears to the users as though each of the n processes has its own processor running at $1/n$ the speed of the real processor
 - Better response time
 - High context switching cost
 - OS actions of saving and restoring a few registers
 - H/W flush: Cache, TLB, branch predictor
- Deciding on the length of the time slice presents a trade-off to a system designer
 - Long enough to amortize the cost of switching
 - without making it so long that the system is no longer responsive. *시간이 response time을 줄여 주면 성능이 좋아질 것임*

*time-slice는 switching으로 인한 overhead를 생기는 만큼 줄여 줄 것임
동시에 response time을 줄여 주면 성능이 좋아질 것임*

Example of RR with Time Quantum = 4

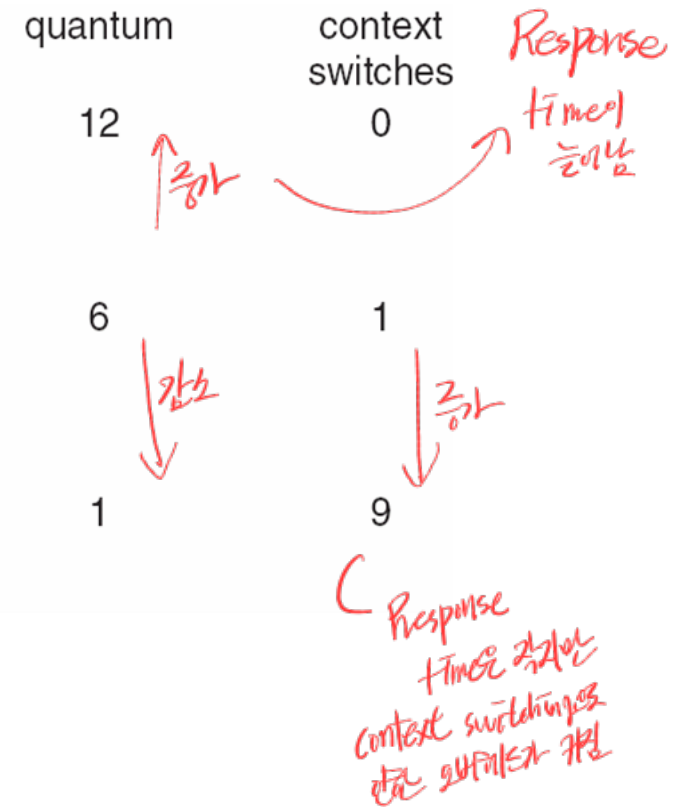
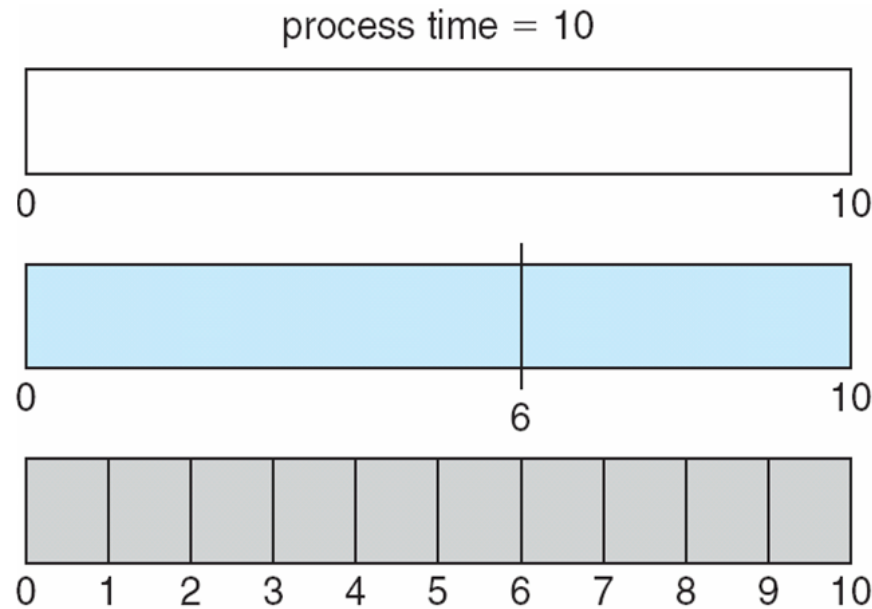
<u>Process</u>	<u>Burst Time</u>
P_1	24
P_2	3
P_3	3

- The Gantt chart is:

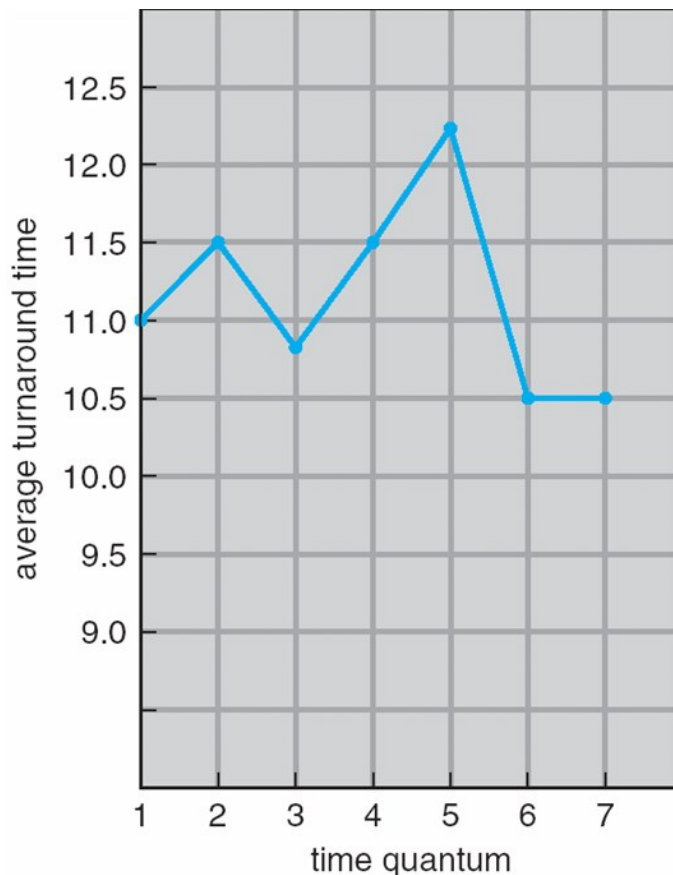


- Typically, higher average turnaround than SJF, but **better response**
 - RR is indeed one of the worst policies if turnaround time is our metric*
- q should be large compared to context switch time
- q usually 10ms to 100ms, context switch < 10 usec

Time Quantum and Context Switch Time



Turnaround Time Varies With The Time Quantum



process	time
P_1	6
P_2	3
P_3	1
P_4	7

80% of CPU bursts should be shorter than q

아니 100%라면
10 unit 80%의 cpu burst를
실행시켜야 함

Incorporating I/O

CPU burst time $\frac{1}{2}$ and other I/O burst 2 고려

- When a job initiates an I/O request, because the currently-running job won't be using the CPU during the I/O; it is blocked waiting for I/O completion
- When the I/O completes, an interrupt is raised, and the OS runs and moves the blocked process back to the ready state.

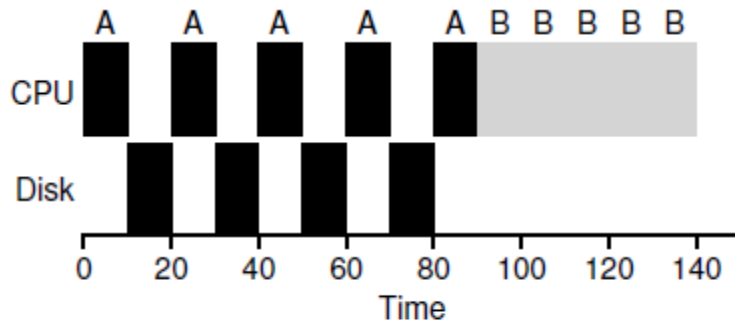


Figure 7.8: Poor Use of Resources

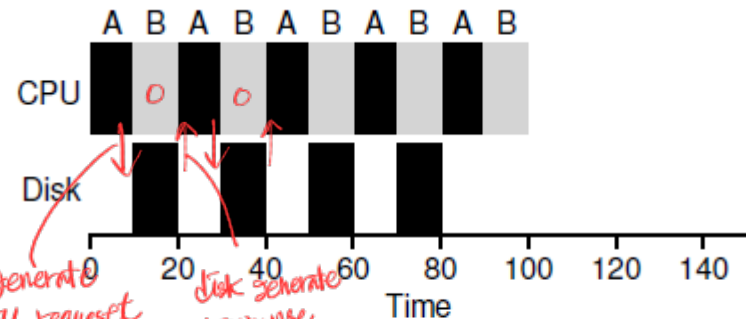


Figure 7.9: Overlap Allows Better Use of Resources

Treat each CPU burst as a job

Scheduling Schemes

- **Priority scheduling**

- Scheduling criteria

- Process priority

- Tie breaking: FCFS

- Priority range is different for each system

*Linux에서는
0~319 사용
(4096)*

- Mapping from the numerical value of the priority to the priority level is different for each system

작은 수를 high로, 큰 수를 high로

→ 0보다 작을, T_{max} 은 작은 수를 ↑

- Can be either preemptive or non-preemptive

- Major problem

- Starvation

- Solution

*실행시간 증가
long process의 priority를 높임*

- **Aging** – as time progresses increase the priority of the process

Scheduling Policies

- **Priority**

- Classification

- **Static priority** (external priority)

- Decided at process creation time and fixed during execution of the process
 - Not adaptable to system environments
 - Simple, low-overhead

- **Dynamic priority** (internal priority)

- Initial priority at process creation time
 - May vary as the state of the system and processes changes
 - Adaptable to system environments
 - Complex, high overhead due to priority adjustment

Example of Priority Scheduling

<u>Process</u>	<u>Burst Time</u>	<u>Priority</u>
P_1	10	3
P_2	1	1
P_3	2	4
P_4	1	5
P_5	5	2

Handwritten diagram showing priority values 1, 2, 3, 4, 5 in a vertical column. Arrows indicate a circular sequence: 1 → 2 → 3 → 4 → 5 → 1. A red arrow labeled "start" points to the value 1, and a red arrow labeled "static" points to the value 2.

- Priority scheduling Gantt Chart

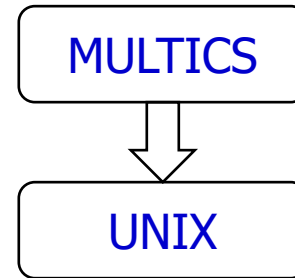


- Average waiting time = 8.2 msec

$$(0 + 1 + 6 + 16 + 18) / 5 = 8.2$$

MLFQ (Multi-Level Feedback Queue)

- First described by Corbato et al. in 1962 in Compatible Time-Sharing System (CTSS) and Multics



Corbato, MIT 1965
(Turing Award 1990)

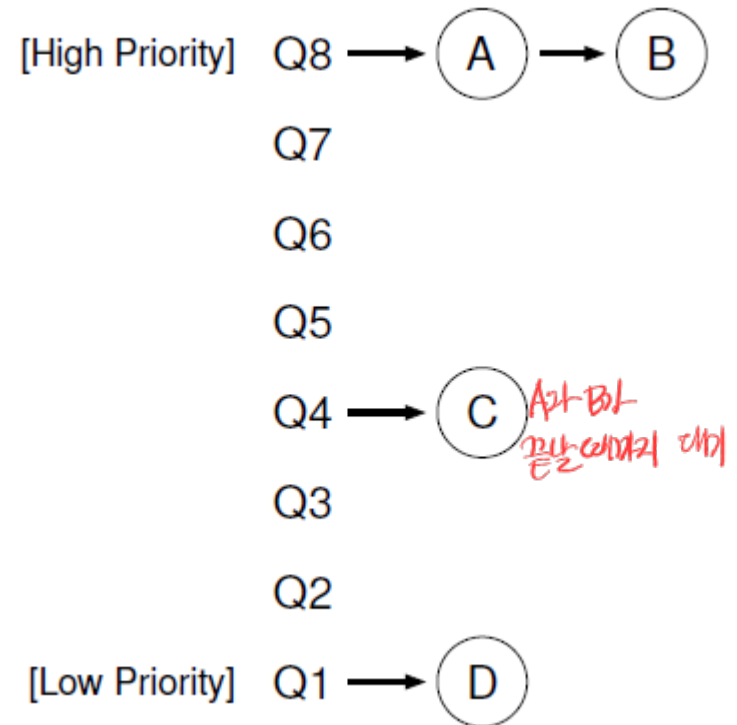
Ken Thompson
Dennis Ritchie
Bell Lab 1973
(Turing Award 1983)

- To optimize turnaround time
 - running shorter jobs first
 - Problem: SJF/STCF cannot know how long a job will run for
- To be responsive to interactive users
 - Round Robin
 - Problem: RR is terrible for turnaround time.
- Our problem
 - Given that we in general do not know anything about a process, how can we build a scheduler to achieve these goals?
 - ➔ learn from the past to predict the future

MLFQ: Basic Rules

- Multiple separate ready queues, each assigned a different priority.
- **Rule 1:** If $\text{Priority}(A) > \text{Priority}(B)$, A runs (B doesn't).
- **Rule 2:** If $\text{Priority}(A) = \text{Priority}(B)$, A & B run in RR.

- Interactive process
 - Repeatedly relinquishes the CPU while waiting for input
 - High priority
- Batch process (CPU-bound)
 - Uses the CPU intensively for long periods of time
 - Low Priority



Attempt #1: How To Change Priority

- **Rule 3:** When a job enters the system, it is placed at the highest priority (the topmost queue). *시작하면 highest로*
- **Rule 4a:** If a job uses up an entire time slice while running, its priority is reduced (i.e., it moves down one queue).
- **Rule 4b:** If a job gives up the CPU before the time slice is up, it stays at the same priority level. *time slice를 다 쓰면 priority ↓*

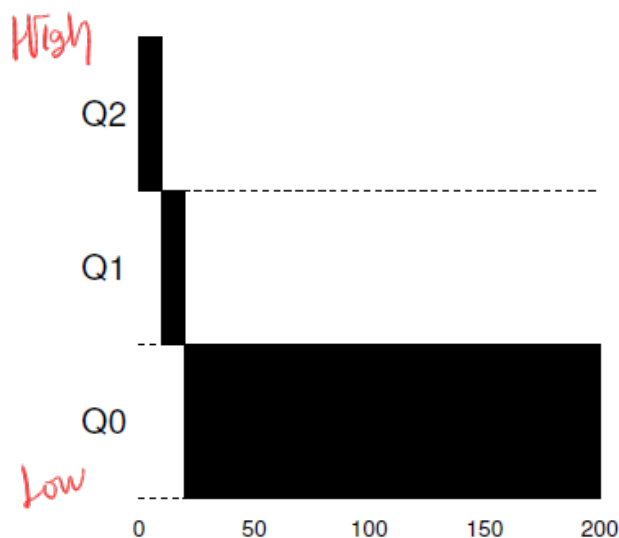


Figure 8.2: Long-running Job Over Time

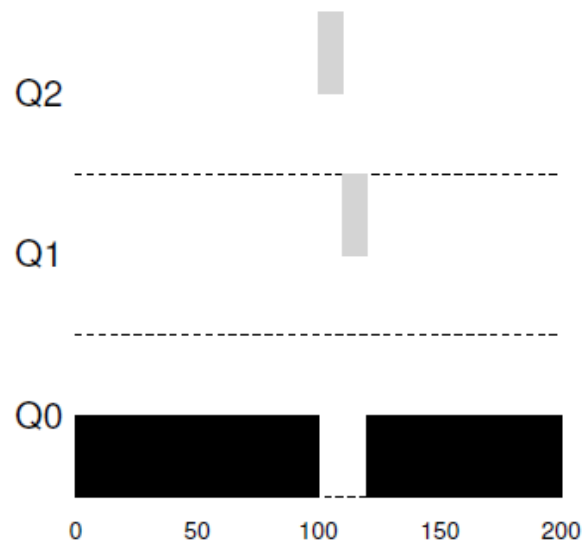


Figure 8.3: Along Came An Interactive Job

Problems With Our Current MLFQ

- **Starvation**

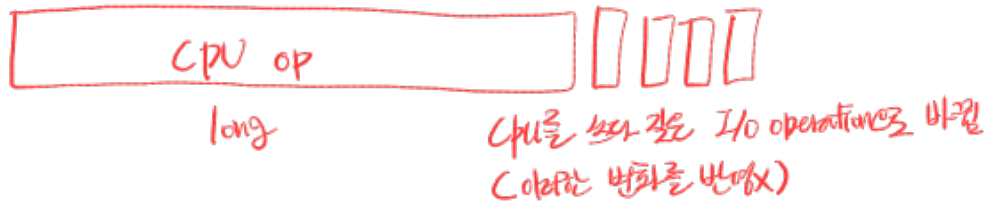
- if there are “too many” interactive jobs in the system, long-running jobs will never receive any CPU time (they starve).

→ Need Priority Boost

- **Gaming the scheduler**

- a smart user could rewrite their program
- before the time slice is over, issue an I/O operation (to some file you don't care about) and thus relinquish the CPU

- Program may **change its behavior** over time



Attempt #2: The Priority Boost

- **Rule 5:** After some time period S , move all the jobs in the system to the topmost queue.
장시간 대기 → priority를 최대로 올림
 - Prevent starvation and detect the change of behavior
- **Aging** is also a choice
 - Processes that have long waiting time moves up in the queue hierarchy

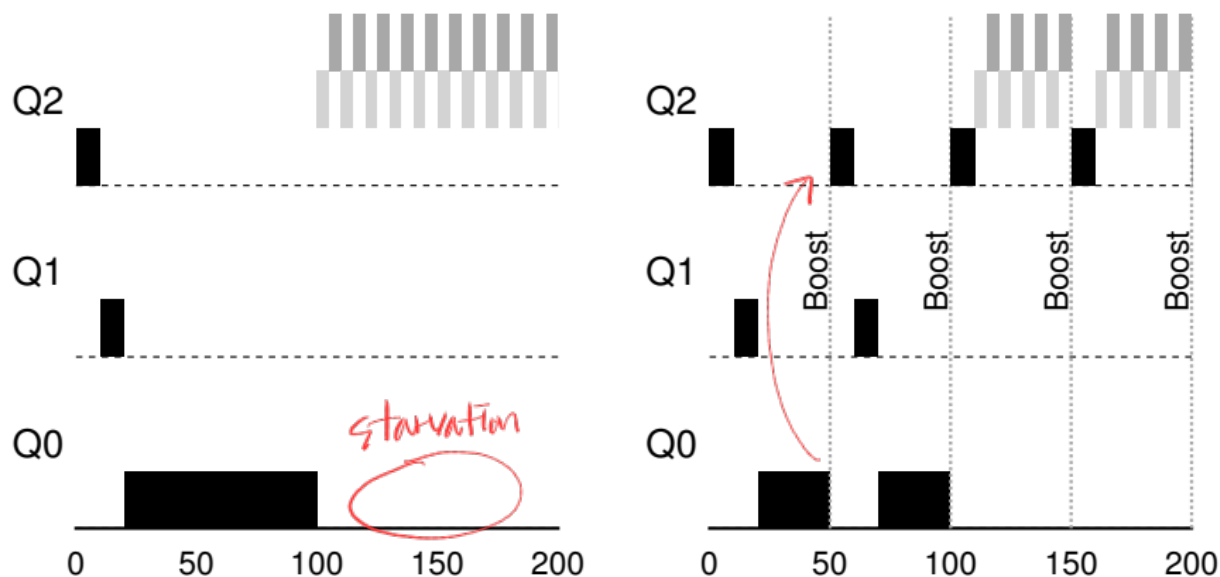


Figure 8.5: Without (Left) and With (Right) Priority Boost

Attempt #3: Better Accounting

- **Rule 4:** Once a job uses up its time allotment at a given level (regardless of how many times it has given up the CPU), its priority is reduced (i.e., it moves down one queue).
알고리즘 이상 쓰면 내림
 - Instead of forgetting how much of a time slice a process used at a given level, the scheduler should keep track

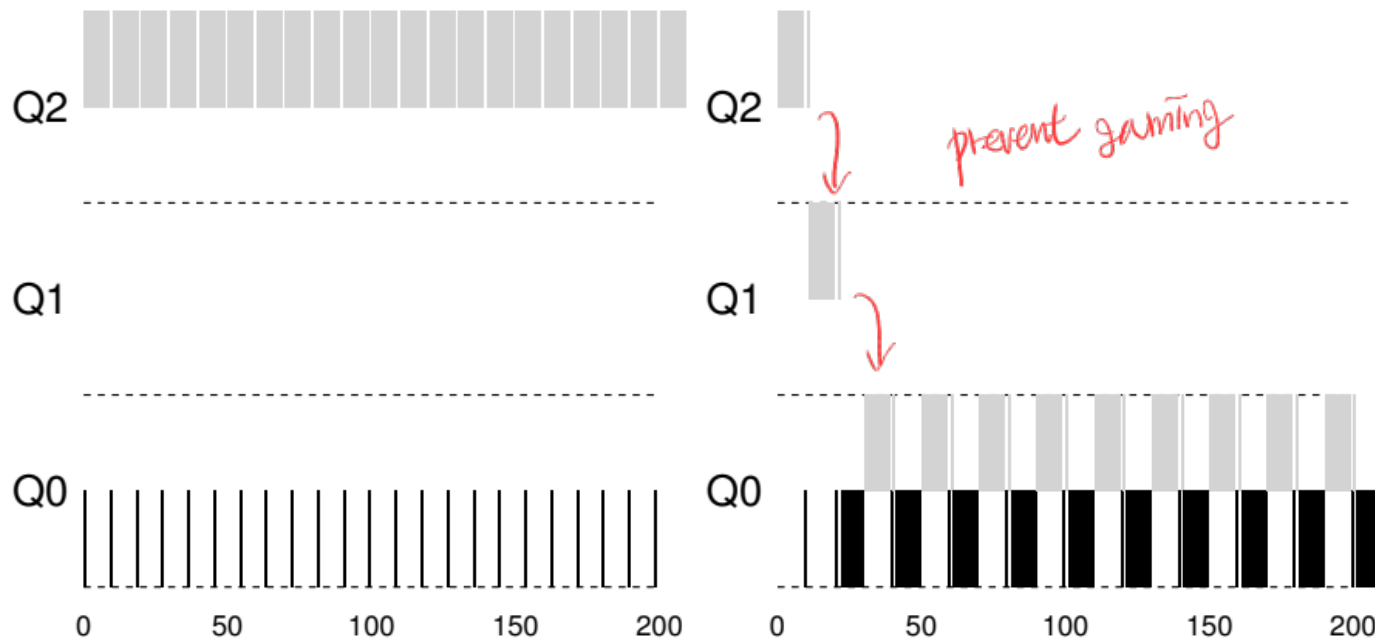


Figure 8.6: Without (Left) and With (Right) Gaming Tolerance

Attempt #4: Different Time Slice

- **Three queues:**

- Q_2 – RR with time quantum 8ms
- Q_1 – RR with time quantum 16ms
- Q_0 – FCFS

- **Scheduling**

- A new job enters queue Q_2
 - When it gains CPU, job receives 8ms
 - If it does not finish in 8ms, job is moved to queue Q_1
- At Q_1 job receives additional 16ms
 - If it still does not complete, it is preempted and moved to queue Q_0

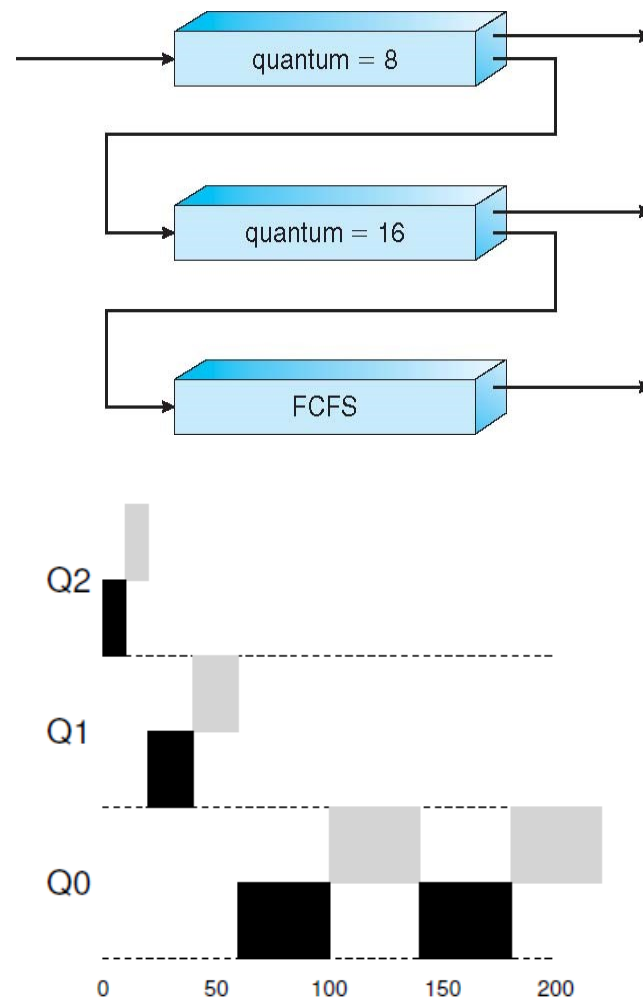


Figure 8.7: Lower Priority, Longer Quanta

Parameters for MLFQ scheduling

- The number of queues
- The scheduling algorithm for each queue
- The time slice of each queue
- The method used to determine when to upgrade a process to a higher-priority queue *priority boost*
- The method used to determine when to demote a process to a lower-priority queue
- The method used to determine which queue a process will enter when that process needs service
- Easy Configuration
 - Provides a set of tables that determine exactly how the priority of a process is altered, how long each time slice is, and how often to boost the priority of a job (Solaris)
 - Uses a formula to calculate the current priority level of a job (FreeBSD)

Quiz

1. FCFS scheduling is preemptive. (X)
2. In Shortest Job First scheduling, the process with the smallest next CPU burst is selected next. (O)
3. Shortest Time-to-Completion First scheduling can preempt a currently running process if a new process arrives with a shorter remaining time. (O)
스킵
4. Round Robin scheduling is particularly effective in reducing the waiting time for computing-intensive processes. (X)
5. Priority scheduling can lead to starvation if not implemented with aging. (O)
6. ~~FCFS~~ scheduling tends to favor CPU-bound processes over I/O-bound processes. (X)
선택 ← *스킵* *안 보게*
CPU 시간이 많은 process
7. Round Robin scheduling involves a fixed time quantum, during which each process gets to execute. (O)
8. In SJF scheduling, predicting the length of the next CPU burst is straightforward and always accurate. (X)

Quiz

9. MLFQ allows processes to move between different priority queues based on their CPU burst behavior. (O)
10. In MLFQ, processes that use more CPU time are moved to ~~higher~~-priority queues. (X)
11. MLFQ is designed to give preference to interactive and short-running processes. (O)
12. Higher-priority queues in MLFQ typically have shorter time quantum than lower-priority queues. (O)
13. A process that ages in MLFQ can be moved back to a higher-priority queue to prevent starvation. (O)
14. MLFQ is an example of a ~~non~~-preemptive scheduling algorithm. (X)
15. MLFQ requires multiple queues with different scheduling algorithms per queue. (O)
16. In MLFQ, all processes within the same queue are allocated the same amount of time quantum. (O)
17. MLFQ can be configured to dynamically adjust priority based on aging mechanisms. (O)
18. MLFQ performs well for both batch and interactive systems. (O)
19. Processes with long CPU bursts may eventually be assigned the lowest priority in MLFQ. (O)
20. MLFQ require some tuning or configuration for different workloads. (O)

Case Studies - Scheduling in Linux

- **O(1) scheduler (~v2.5)**

- Preemptive, priority based (similar to the MLFQ)
 - Two priority ranges: time-sharing and real-time
 - Each processor maintains its own runqueue and schedules itself independently
- Fairness
- Time slice is allocated according to priority
 - If $(\text{priority} < 120) (140 - \text{priority}) * 20$, else $(140 - \text{priority}) * 5$
 - Ex. Priority: 100 $\rightarrow$ 800ms, Priority: 120 $\rightarrow$ 100ms, Priority: 139 $\rightarrow$ 5ms
- Dynamic priority
 - $\max(100, \min(\text{static priority} - \text{bonus}, 139))$
 - bonus(-5~5) is given based on the average sleep time
- As soon as the allocated execution time is used up, the task is placed in the expired queue.
- When all tasks at the lowest priority level have run, the expired and active queues at that priority level are exchanged.

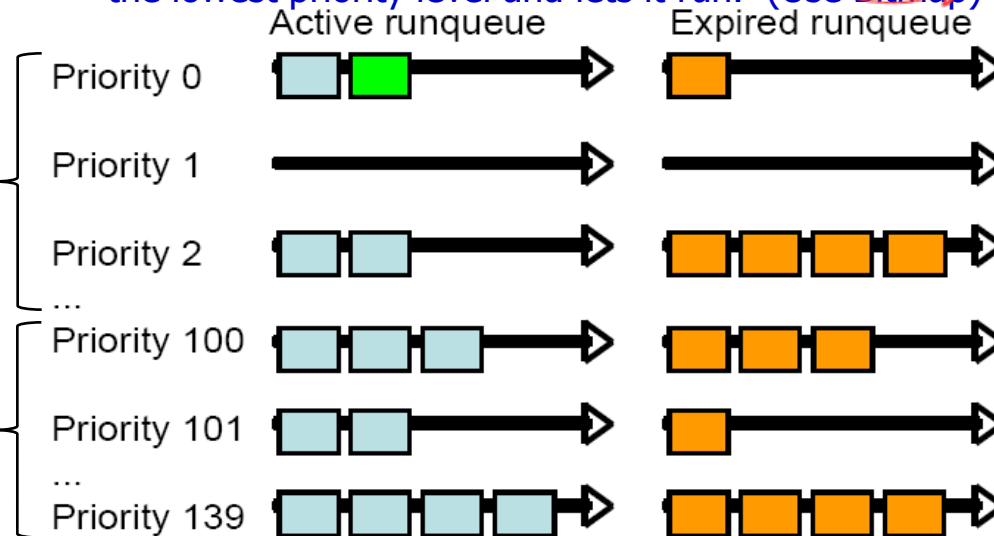
O(1) scheduling

The active queue contains the tasks that are waiting to run.

real-time tasks

normal tasks
(nice: -20~19)

The scheduler picks the first task in the active queue of the lowest priority level and lets it run. (Use Bitmap)



The expired queue contains the tasks that have recently run and therefore must wait for the other queue to be empty.

140 priority levels (lower numbers indicate higher priority);
The top 40: normal user tasks, The lower 100: real-time tasks.